

Deploying a node.js application on AWS Ec2 instance

Introduction

This project demonstrates how to host a Node.js application on an AWS EC2 instance.

Note: When deploying a Node.js application, you don't need an additional web server—Node.js itself can act as the web server.

The steps are written in a simple way so that anyone can follow them easily. All you need is an EC2 instance, Node.js as the web server, and npm as the package manager.

Requirements

1. AWS account with an EC2 instance (Linux-based)
2. SSH key pair to connect to the instance
3. Node.js and npm installed on EC2
4. Security group with HTTP (port 80) or app port open
5. Your Node.js application code.

Deployment Steps:

Step 1: Launch an EC2 Instance

Go to AWS Management Console --> EC2 --> Launch instance (named as NodeApp)

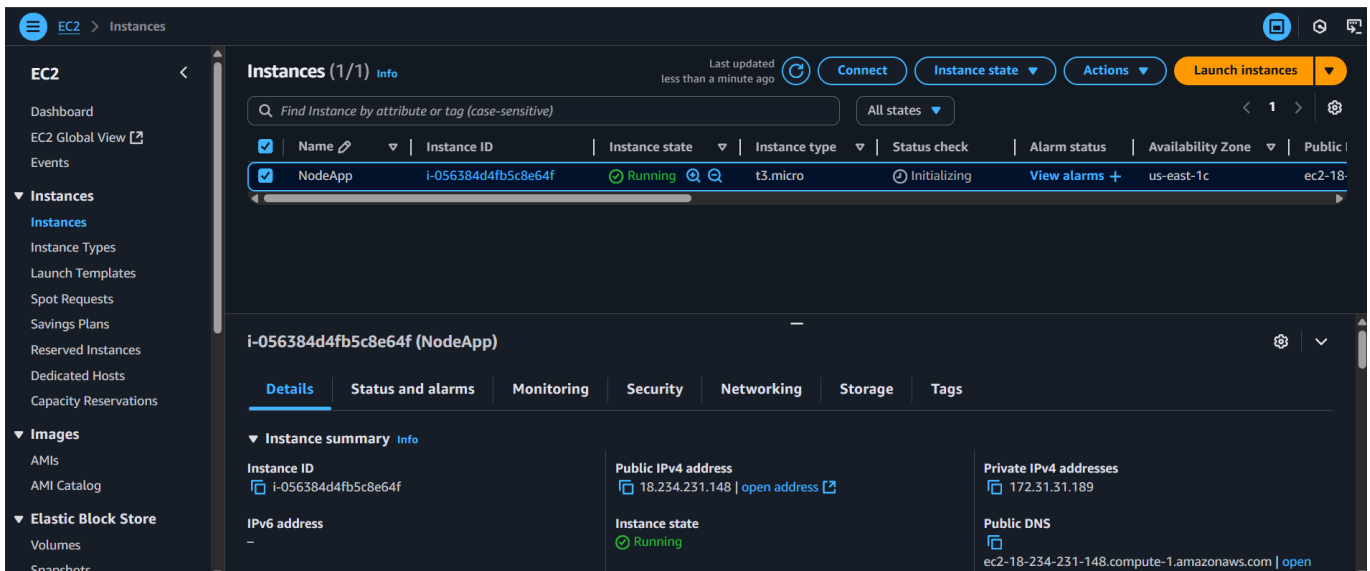
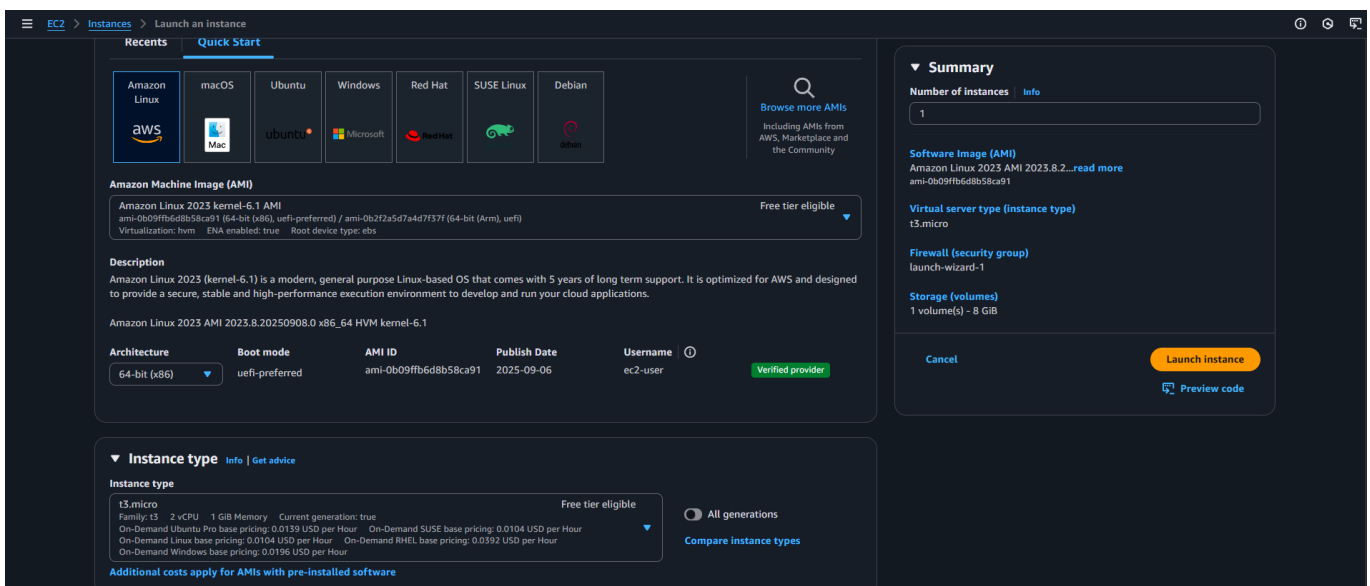
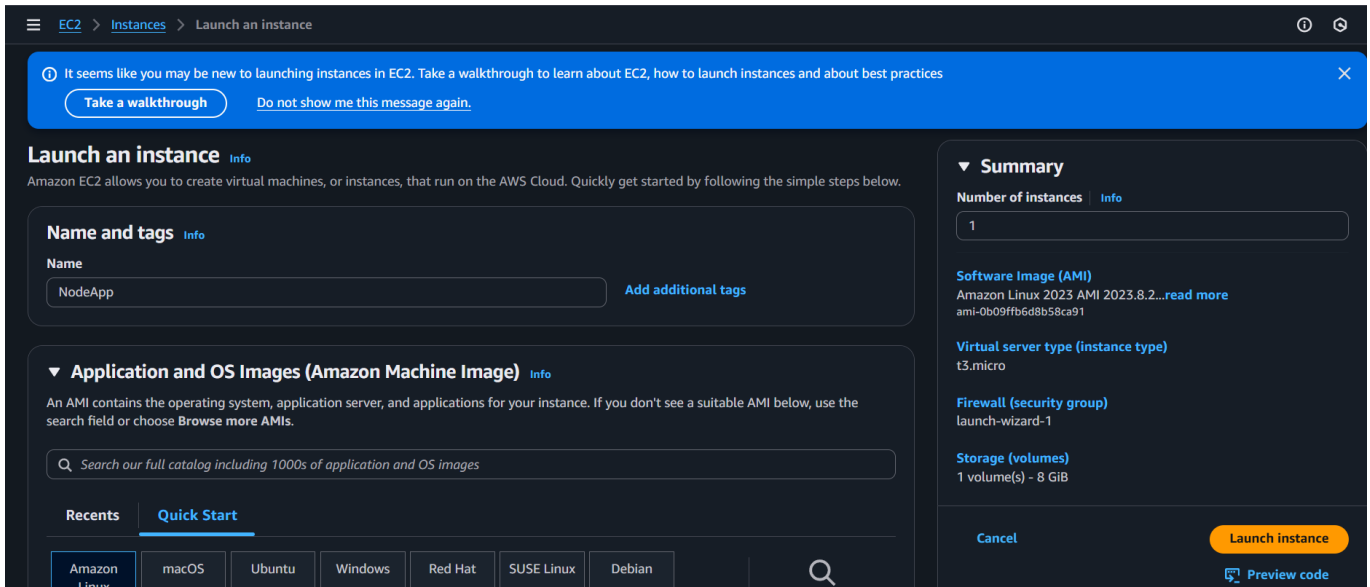
Choose Amazon Linux AMI

Select instance type(t3.micro for free tier)

Configure security group --> Allow HTTP(80),HTTPS(443),SSH(22)

Click Launch Instance = Wait until the status shows running.

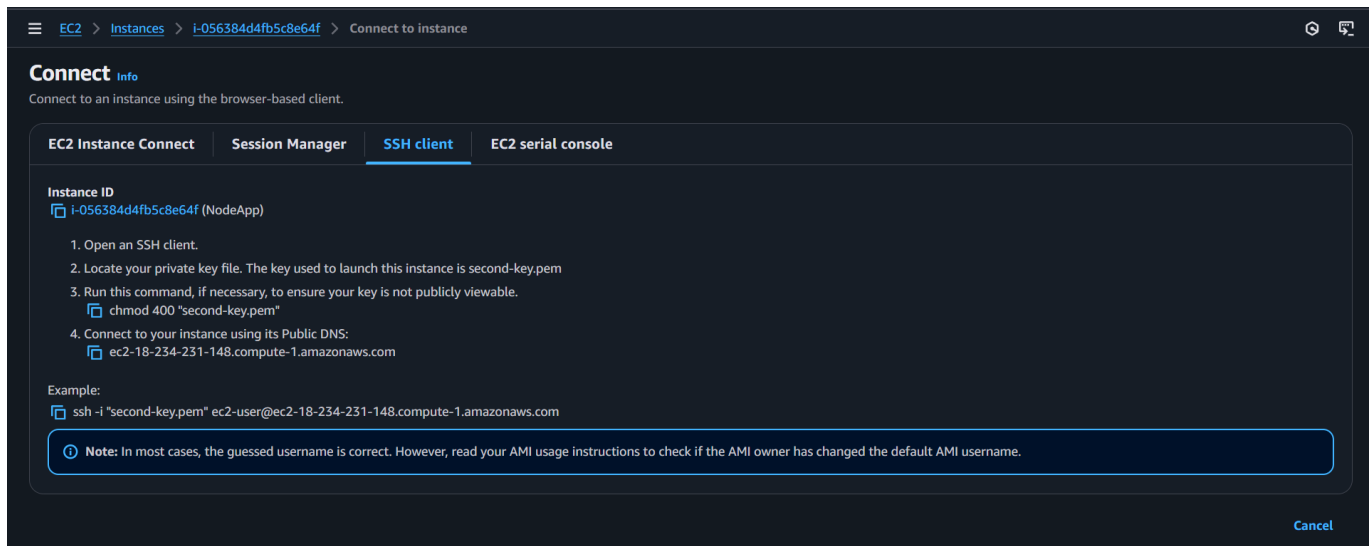
Creating and instance.



Step 2: Connecting to EC2 instance

Run this command on terminal-

```
ssh -i "your-key-pem" ec2-user@your-public-ip
```



Step 3: Updating Packages

Run this command on terminal-

```
sudo yum update
```

```
ASUS@LAPTOP-20439K13 MINGW64 /d/Shubham_workspace/ssh-key
$ ssh -i "second-key.pem" ec2-user@ec2-18-234-231-148.compute-1.amazonaws.com
The authenticity of host 'ec2-18-234-231-148.compute-1.amazonaws.com (18.234.231.148)' can't be established.
ED25519 key fingerprint is SHA256:BZavLi5bfapskj1+L2irxcDXqBrhlMLdfnDNQr8xOc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-234-231-148.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
```

The terminal window shows the user's prompt as 'ASUS@LAPTOP-20439K13 MINGW64'. They run the command '\$ ssh -i "second-key.pem" ec2-user@ec2-18-234-231-148.compute-1.amazonaws.com'. The terminal displays the standard SSH warning about the host's authenticity, showing the ED25519 key fingerprint. The user responds with 'yes'. A warning message indicates that the host has been permanently added to the list of known hosts. Below this, there is a large ASCII art logo for Amazon Linux 2023, which includes the text 'Amazon Linux 2023' and the URL 'https://aws.amazon.com/linux/amazon-linux-2023'. After the logo, the terminal shows the user at the prompt '[ec2-user@ip-172-31-31-189 ~]\$' running the command '\$ sudo yum update'. The output of this command shows 'Amazon Linux 2023 Kernel Livepatch repository' being installed, with dependencies resolved. At the bottom right of the terminal window, there is a status bar displaying '216 kB/s | 21 kB' and a timer '00:00'.

```
#_
#####      Amazon Linux 2023
#####
###|
#/_
V~' '->
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-31-189 ~]$ sudo yum update
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-31-189 ~]$
```

216 kB/s | 21 kB 00:00

Step 4: Installing Node.js as a webserver and NPM as a package manager

```
sudo yum install nodejs -y
```

```
sudo yum install npm -y
```

```
[ec2-user@ip-172-31-189 ~]$ sudo yum install nodejs -y
Last metadata expiration check: 0:00:50 ago on Sat Sep 13 13:16:21 2025.
Dependencies resolved.

=====
Package                                Architecture      Version                                Repository          Size
=====
Installing:
nodejs                                x86_64             1:18.20.8-1.amzn2023.0.2             amazonlinux          13 M
Installing dependencies:
libbrotli                             x86_64             1.0.9-4.amzn2023.0.2                 amazonlinux          315 k
nodejs-libs                           x86_64             1:18.20.8-1.amzn2023.0.2             amazonlinux          14 M
Installing weak dependencies:
nodejs-docs                           noarch             1:18.20.8-1.amzn2023.0.2             amazonlinux          7.8 M
nodejs-full-i18n                      x86_64             1:18.20.8-1.amzn2023.0.2             amazonlinux          8.4 M
nodejs-npm                            x86_64             1:10.8.2-1.18.20.8.1.amzn2023.0.2    amazonlinux          1.9 M
=====

Transaction Summary
=====
Install 6 Packages

Total download size: 45 M

[ec2-user@ip-172-31-189 ~]$ sudo yum install npm -y
Last metadata expiration check: 0:01:56 ago on Sat Sep 13 13:16:21 2025.
Package nodejs-npm-1:10.8.2-1.18.20.8.1.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-189 ~]$
```

Step 5: Check Versions

node --version

npm --version

```
[ec2-user@ip-172-31-31-189 ~]$ node --version
v18.20.8
[ec2-user@ip-172-31-31-189 ~]$

[ec2-user@ip-172-31-31-189 ~]$ npm --version
10.8.2
[ec2-user@ip-172-31-31-189 ~]$
```

Step 6: Create Directory

mkdir nodeapp

```
[ec2-user@ip-172-31-31-189 ~]$ mkdir nodeapp
[ec2-user@ip-172-31-31-189 ~]$ ls
nodeapp
[ec2-user@ip-172-31-31-189 ~]$
```

Step 7: Verify Directory

ls

Step 8: Go Inside Directory

cd nodeapp/

ls

```
[ec2-user@ip-172-31-31-189 ~]$ cd nodeapp/
[ec2-user@ip-172-31-31-189 nodeapp]$ ls
[ec2-user@ip-172-31-31-189 nodeapp]$
```

Step 9: Check Git Installation

git --version

If not installed:

sudo yum install git

```
[ec2-user@ip-172-31-31-189 ~]$ sudo yum install git
Last metadata expiration check: 0:07:03 ago on Sat Sep 13 13:16:21 2025.
Dependencies resolved.
=====
Package                               Architecture      Version           Repository        Size
=====
Installing:
git                                   x86_64            2.50.1-1.amzn2023.0.1  amazonlinux      53 k
Installing dependencies:
git-core                             x86_64            2.50.1-1.amzn2023.0.1  amazonlinux      4.9 M
git-core-doc                         noarch            2.50.1-1.amzn2023.0.1  amazonlinux      2.8 M
perl-Error                           noarch            1:0.17029-5.amzn2023.0.2  amazonlinux      41 k
perl-File-Find                       noarch            1.37-477.amzn2023.0.7    amazonlinux      25 k
perl-Git                             noarch            2.50.1-1.amzn2023.0.1    amazonlinux      41 k
perl-TermReadKey                     x86_64            2.38-9.amzn2023.0.2     amazonlinux      36 k
perl-lib                             x86_64            0.65-477.amzn2023.0.7    amazonlinux      15 k
Transaction Summary
=====
Install 8 Packages

Total download size: 7.9 M
Installed size: 41 M
Is this ok [y/N]: y
Downloading Packages:
(1/8): git-2.50.1-1.amzn2023.0.1.x86_64.rpm           1.8 MB/s | 53 kB   00:00
(2/8): perl-Error-0.17029-5.amzn2023.0.2.noarch.rpm   1.8 MB/s | 41 kB   00:00
(3/8): git-core-doc-2.50.1-1.amzn2023.0.1.noarch.rpm  40 MB/s | 2.8 MB   00:00
(4/8): perl-File-Find-1.37-477.amzn2023.0.7.noarch.rpm 1.1 MB/s | 25 kB   00:00
(5/8): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm     48 MB/s | 4.9 MB   00:00
```

Step 10: Check Git Version

git --version

```
[ec2-user@ip-172-31-31-189 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-31-189 ~]$
```

Step 11: Clone Your Application Code

sudo git clone " your-repository-link"

```
[ec2-user@ip-172-31-31-189 ~]$ sudo git clone https://github.com/iamtruptime/node-app.git
Cloning into 'node-app'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
[ec2-user@ip-172-31-31-189 ~]$
```

Step 12: Go to Cloned Project

ls

cd node-app/

```
[ec2-user@ip-172-31-31-189 ~]$ ls
node-app  nodeapp
[ec2-user@ip-172-31-31-189 ~]$
```

Step 13: Verify Files

ls

Step 14: Check for package.json and app.js

Ignore Dockerfile if present.

```
[ec2-user@ip-172-31-31-189 ~]$ cd node-app/
[ec2-user@ip-172-31-31-189 node-app]$ ls
Dockerfile  app.js  package.json
[ec2-user@ip-172-31-31-189 node-app]$
```

Step 15: View package.json

cat package.json

```
[ec2-user@ip-172-31-31-189 node-app]$ cat package.json
{
  "name": "node-app",
  "version": "1.0.0",
  "description": "Simple Node.js app",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
[ec2-user@ip-172-31-31-189 node-app]$ sudo npm install
added 69 packages, and audited 70 packages in 3s
14 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.6.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.6.0
npm notice To update run: npm install -g npm@11.6.0
npm notice
[ec2-user@ip-172-31-31-189 node-app]$
```

Step 16: Install Dependencies

```
sudo npm install
```

Step 17: Check node_modules Folder

If present, all dependencies are installed successfully.

```
[ec2-user@ip-172-31-31-189 node-app]$ ls
Dockerfile app.js node_modules package-lock.json package.json
[ec2-user@ip-172-31-31-189 node-app]$
```

Step 18: View app.js File

```
cat app.js
```

```
[ec2-user@ip-172-31-31-189 node-app]$ cat app.js
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello, this is a Node.js application running in a Docker container!');
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
[ec2-user@ip-172-31-31-189 node-app]$
```

Step 19: Run Application

```
node app.js
```

You should see:

Server is running...

```
[ec2-user@ip-172-31-31-189 node-app]$ node app.js
Server is running on http://localhost:3000
```

Step 20: Configure Security Group

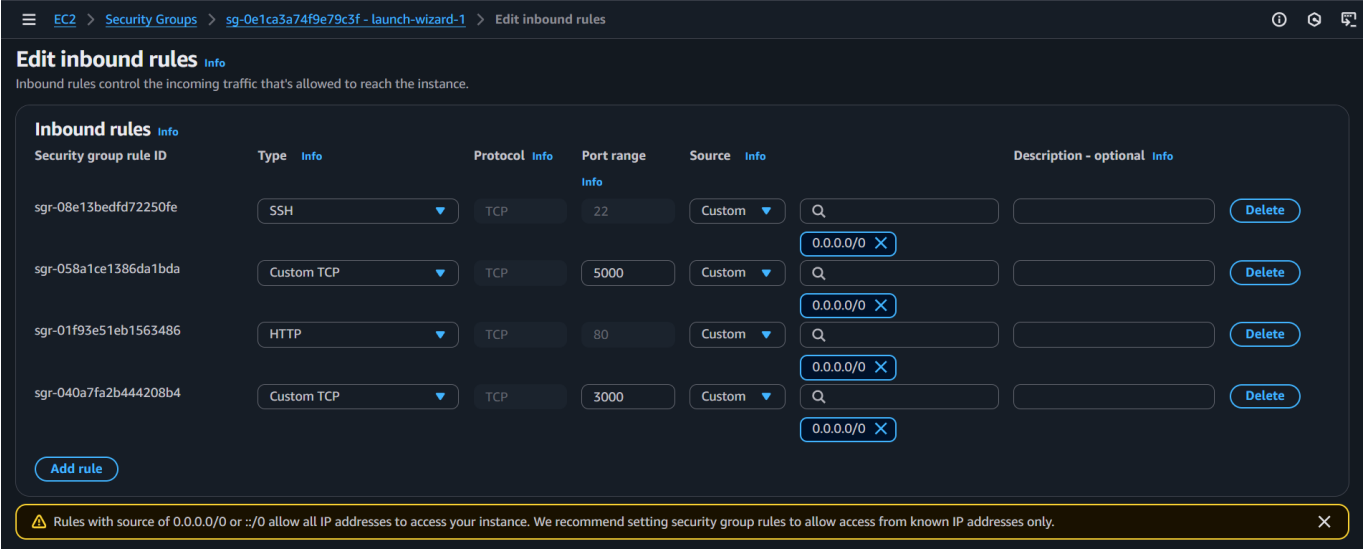
Go to AWS Security Group settings

Add a new inbound rule:

Type: Custom TCP

Port: 3000

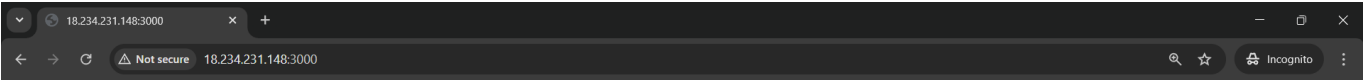
Source: Anywhere (IPv4)



Step 21: Access Application

Copy your EC2 Public IP and open it in browser with port 3000:

http://"public-ip":3000



Hello, this is a Node.js application running in a Docker container!

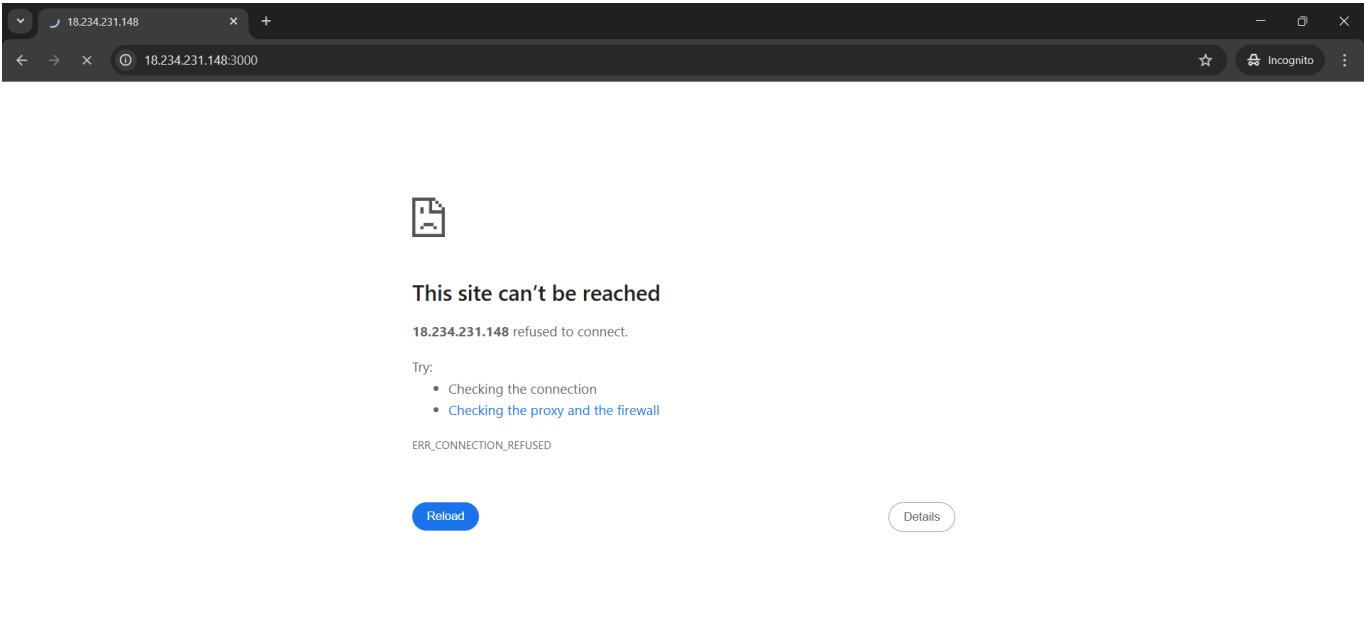
Step 22: Run Application in Background (Optional)

If you want your app to keep running after closing the terminal:

```
sudo npm install -g pm2
```

```
pm2 start app.js
```

Now refresh your browser — your app will still be running.



Hello, this is a Node.js application running in a Docker container!

This is final output of your project.